

Introduction to Machine Learning Assignment1

Gabriel Severiche · 14 de septiembre de 2026



Contenido

Khipus.ai

Introduction to Machine Learning

Data Preparation

Assignment 1

Name : Gabriel Severiche

Assignment Instructions

1. Import the Titanic dataset
 2. Data Exploration
 3. Data Cleaning (addressing missing values)
- Feature Selection
5. Splitting Training and Test Data

Khipus.ai

Introduction to Machine Learning

Data Preparation

© Copyright Notice 2025, Khipus.ai - All Rights Reserved.

Assignment 1

Name : Gabriel Severiche

Assignment Instructions

Using the Titanic dataset provided:

1. Import the Titanic.csv file into a pandas dataframe
2. Perform a detailed data exploration.
3. Clean the dataset by addressing missing values.
4. Select relevant features for predicting survival.
5. Split the dataset into training and testing sets (80%-20% split).

1. Import the Titanic dataset

```
In [1]:  
# Import necessary Libraries  
import pandas as pd  
from sklearn.model_selection import train_test_split
```

```
In [2]:  
# Your code here  
  
# Load the Titanic dataset  
  
df=pd.read_csv('titanic.csv')
```

2. Data Exploration

Before working with the dataset, it's important to understand its structure, data types, and summary statistics. Below are examples of how to explore the data.

In [3]:

```
# Your code here  
  
# Display the first few rows of the dataset  
  
df.head()
```

Out [3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helg...	female	22.0	1	1	3101298	12.2875	NaN	S

In [4]:

```
# Display summary statistics  
df.info()  
df.describe()
```

Out [4]:

```
<class 'pandas.DataFrame'>  
RangeIndex: 418 entries, 0 to 417  
Data columns (total 12 columns):  
#   Column      Non-Null Count  Dtype  
---  ---      -  
0   PassengerId  418 non-null    int64  
1   Survived     418 non-null    int64  
2   Pclass       418 non-null    int64  
3   Name         418 non-null    str  
4   Sex          418 non-null    str  
5   Age          332 non-null    float64  
6   SibSp        418 non-null    int64  
7   Parch        418 non-null    int64  
8   Ticket       418 non-null    str  
9   Fare         417 non-null    float64  
10  Cabin        91 non-null     str  
11  Embarked     418 non-null    str  
dtypes: float64(2), int64(5), str(5)  
memory usage: 39.3 KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	418.000000	418.000000	418.000000	332.000000	418.000000	418.000000	417.000000
mean	1100.500000	0.363636	2.265550	30.272590	0.447368	0.392344	35.627188
std	120.810458	0.481622	0.841838	14.181209	0.896760	0.981429	55.907576
min	892.000000	0.000000	1.000000	0.170000	0.000000	0.000000	0.000000
25%	996.250000	0.000000	1.000000	21.000000	0.000000	0.000000	7.895800
50%	1100.500000	0.000000	3.000000	27.000000	0.000000	0.000000	14.454200
75%	1204.750000	1.000000	3.000000	39.000000	1.000000	0.000000	31.500000
max	1309.000000	1.000000	3.000000	76.000000	8.000000	9.000000	512.329200

In [5]:

```
# Check for missing values  
df.isnull().sum()
```

Out [5]:

```
PassengerId    0  
Survived       0  
Pclass         0  
Name           0
```

```
Sex          0
Age          86
SibSp        0
Parch        0
Ticket       0
Fare         1
Cabin       327
Embarked     0
dtype: int64
```

3. Data Cleaning (addressing missing values)

Real-world datasets often contain missing values, duplicate rows, or incorrect data. Cleaning the data ensures its quality and usability for analysis.

```
In [6]:
data_cleaned = df.copy()

# Fill missing 'Age' with the median
data_cleaned['Age'] = data_cleaned['Age'].fillna(data_cleaned['Age'].median())

# Fill missing 'Embarked' with the mode
data_cleaned['Embarked'] = data_cleaned['Embarked'].fillna(data_cleaned['Embarked'].mode()[0])

# Fill missing 'Fare' with the median
data_cleaned['Fare'] = data_cleaned['Fare'].fillna(data_cleaned['Fare'].median())

# Drop 'Cabin' as it has too many missing values
data_cleaned = data_cleaned.drop('Cabin', axis=1)

# Check for remaining missing values
data_cleaned.isnull().sum()
```

```
Out [6]:
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

Feature Selection

To build an effective machine learning model, selecting relevant features is crucial. Select relevant features for predicting survival.

```
In [7]:
# Define the features you want to use
# Select features and target
# Your code here
# Select the features from the DataFrame
# Select the target variable from the DataFrame
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
target = 'Survived'
data_selected = data_cleaned[features + [target]]
data_selected.head()
```

Out [7]:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
0	3	male	34.5	0	0	7.8292	Q	0
1	3	female	47.0	1	0	7.0000	S	1
2	2	male	62.0	0	0	9.6875	Q	0
3	3	male	27.0	0	0	8.6625	S	0

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
4	3	female	22.0	1	1	12.2875	S	1

5. Splitting Training and Test Data

In [8]:

```
# Your code here
# Split into training and test sets
x = data_selected[features]
y = data_selected[target]
```

In [11]:

```
# Your code here
# Display basic information about the dataset X_train
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Out [11]:

```
((334, 7), (84, 7), (334,), (84,))
```

In [12]:

```
# Your code here
# Display basic information about the dataset X_test
X_train.info()
```

Out [12]:

```
<class 'pandas.DataFrame'>
Index: 334 entries, 336 to 102
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Pclass      334 non-null    int64
1   Sex         334 non-null    str
2   Age         334 non-null    float64
3   SibSp       334 non-null    int64
4   Parch       334 non-null    int64
5   Fare        334 non-null    float64
6   Embarked    334 non-null    str
dtypes: float64(2), int64(3), str(2)
memory usage: 20.9 KB
```

In [13]:

```
X_test.info()
```

Out [13]:

```
<class 'pandas.DataFrame'>
Index: 84 entries, 321 to 66
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Pclass      84 non-null    int64
1   Sex         84 non-null    str
2   Age         84 non-null    float64
3   SibSp       84 non-null    int64
4   Parch       84 non-null    int64
5   Fare        84 non-null    float64
6   Embarked    84 non-null    str
dtypes: float64(2), int64(3), str(2)
memory usage: 5.2 KB
```

In [14]:

```
y_train.info()
```

Out [14]:

```
<class 'pandas.Series'>
Index: 334 entries, 336 to 102
Series name: Survived
```

```
Non-Null Count  Dtype
-----
334 non-null   int64
dtypes: int64(1)
memory usage: 5.2 KB
```

```
In [15]:
```

```
y_test.info()
```

```
Out [15]:
<class 'pandas.Series'>
Index: 84 entries, 321 to 66
Series name: Survived
Non-Null Count  Dtype
-----
84 non-null     int64
dtypes: int64(1)
memory usage: 1.3 KB
```